

OEIS A105020 and the binary Goldbach conjecture: an elementary equivalence and its Lean 4 formalization

Federico Di Candia

12 September 2026

Abstract

The formal-conjectures repository contains `0eisA105020.conjecture`, a Lean formalization of a 2007 comment on OEIS A105020 that presents it as “a Goldbach Conjecture for this sequence”. We give an elementary proof that this formal statement is equivalent to the repository’s statement of the binary Goldbach conjecture, including an index lemma needed because the formal statement quantifies over arbitrary indices, and a Lean 4 formalization of every result stated here, checked with `comparator`. No novelty is claimed for the mathematical content: the underlying correspondence is implicit in a 2021 formula of W. I. Hurt in OEIS A045917 (Section 6).

1 Definitions and statements

Throughout, $T_c = c(c+1)/2$. The sequence A105020 [2] is the array whose row $n \geq 0$ contains the numbers $m^2 - n^2$ for $m \geq n+1$, read by upward antidiagonals. The repository [1] defines it as follows (file `FormalConjectures/OEIS/105020.lean`):

```
def triangularNumber (c : ℕ) : ℕ := Nat.choose (c + 1) 2

def antidiagonalIndex (n : ℕ) : ℕ :=
  (Nat.sqrt (8 * n + 1) - 1) / 2

def a (n : ℕ) : ℕ :=
  let c := antidiagonalIndex n
  let k := n - triangularNumber c
  let i := c - k
  let m := c + 1
  m^2 - i^2
```

Thus `triangularNumber c` is T_c , and `a` is indexed from 0, with $a(0) = 1$. This agrees with the current offset of the OEIS entry, whose b-file also starts at $a(0) = 1$.

The comment on A105020 reads:

A “Goldbach Conjecture” for this sequence: when there are n terms between consecutive odd integers $(2n+1)$ and $(2n+3)$ for $n > 0$, at least one will be the product of 2 primes (not necessarily distinct). — M. Hiebl, Jul 15 2007

The comment continues with an example for $n = 3$ that names $a(7) = 7$ and $a(11) = 9$, with the intermediate terms $a(8) = 12$, $a(9) = 15$ and $a(10) = 16$. These indices are shifted by one with respect to the entry’s current offset: in the b-file, and in the repository’s `a`, the same terms are $a(6), \dots, a(10)$. The formal statement below involves only values and differences of indices, so the shift does not affect it (a machine check is in Appendix D).

Its formalization in the repository:

```

@[category research open, AMS 11]
theorem conjecture :
  ∀ (n i j : ℕ), 1 ≤ n →
    a i = 2 * n + 1 →
    a j = 2 * n + 3 →
    j = i + n + 1 →
    ∃ (k : ℕ),
      i < k ∧
      k < j ∧
      (a k).IsSemiprime := by
sorry

```

Here `Nat.IsSemiprime n` is Mathlib's `Nat.IsAlmostPrime 2 n`, that is, $n \neq 0$ and $\Omega(n) = 2$, where $\Omega(n)$ is the number of prime factors of n counted with multiplicity.

The binary Goldbach conjecture in the repository (file `FormalConjectures/Wikipedia/GoldbachConjecture.lean`):

```

@[category research open, AMS 11]
theorem goldbach :
  answer(sorry) ↔ ∀ n : ℕ, 2 < n → Even n → ∃ p q, Prime p ∧ Prime q ∧ n = p + q := by
sorry

```

With the repository's default option, `answer(sorry)` elaborates to `True`. This, and the exact elaborated form of both statements, is checked in Section 5.3; see Section 7 for a build-configuration caveat.

2 Indices on antidiagonals

Lemma 1 (Index decomposition). *Every $N \geq 0$ can be written in exactly one way as $N = T_c + k$ with $0 \leq k \leq c$. Moreover, if $N = T_c + k$ with $0 \leq k \leq c$, then c is `antidiagonalIndex N`.*

Proof. Let $t = \lfloor \sqrt{8N+1} \rfloor$ and $c = \lfloor (t-1)/2 \rfloor$, so that $2c+1 \leq t \leq 2c+2$. From $8T_c+1 = (2c+1)^2 \leq t^2 \leq 8N+1$ we get $T_c \leq N$, and from $8N+1 < (t+1)^2 \leq (2c+3)^2 = 8T_{c+1}+1$ we get $N < T_{c+1} = T_c + c + 1$. Conversely, if $N = T_c + k$ with $0 \leq k \leq c$, then

$$(2c+1)^2 \leq 8N+1 = (2c+1)^2 + 8k < (2c+3)^2,$$

so $\lfloor \sqrt{8N+1} \rfloor \in \{2c+1, 2c+2\}$ and therefore $\lfloor (\lfloor \sqrt{8N+1} \rfloor - 1)/2 \rfloor = c$, which is the value of `antidiagonalIndex N`. In particular c , and hence $k = N - T_c$, is determined by N . $\square$

Lemma 2 (Closed form). *For $0 \leq k \leq c$,*

$$a(T_c + k) = (c+1)^2 - (c-k)^2 = (k+1)(2c+1-k).$$

With $s = c+1$ and $d = k+1$ this reads $a(T_c + k) = d(2s-d)$. In particular $a(T_n) = 2n+1$ for every $n \geq 0$.

Proof. By Lemma 1 the antidiagonal index of $T_c + k$ is c , so the local variables in the definition of `a` take the values $c, k, i = c - k$ and $m = c + 1$. Since $k \leq c < c + 1$, none of the subtractions on $\mathbb{N}$ truncates, and

$$a(T_c + k) = (c+1)^2 - (c-k)^2 = (k+1)(2c+1-k).$$

Taking $k = 0$ gives $a(T_n) = 2n + 1$. $\square$

Remark. The indices at which a takes a given value v correspond to the factorizations of v : $a(N) = v$ if and only if there are integers d, e with $1 \leq d \leq e$, $d \equiv e \pmod{2}$ and $v = de$, such that $N = T_{(d+e)/2-1} + (d-1)$. Different such factorizations give different indices, by the uniqueness in Lemma 1. In particular an odd value occurs once for every factorization into two factors $d \leq e$, and not only at the start of an antidiagonal.

3 The index lemma

Lemma 3. *Let $n \geq 1$, and suppose that $a(i) = 2n + 1$, $a(j) = 2n + 3$ and $j = i + n + 1$. Then $i = T_n$ and $j = T_{n+1}$.*

Proof. Put $u = 2n + 1$. By Lemma 1, write $i = T_s - 1 - r$ with $s \geq 1$ and $0 \leq r \leq s - 1$ (in the notation of Lemma 1, $s = c + 1$ and $r = c - k$), and put $d = s - r$, so that $1 \leq d \leq s$. Similarly, write $j = T_{s'} - 1 - r'$ and put $d' = s' - r' \geq 1$. By Lemma 2,

$$u = s^2 - r^2 = d(2s - d), \quad u + 2 = s'^2 - r'^2 = d'(2r' + d').$$

Since $3 \leq u \leq s^2$, we have $s \geq 2$.

Step 1. Expanding with $s = r + d$ and $s' = r' + d'$ gives the polynomial identity

$$2(j - i) - (u + 1) = [r'(r' - 1) + d' - s(s - 1) - 2d + 1] + [d'(2r' + d') - d(2s - d) - 2].$$

The second bracket vanishes because $a(j) = u + 2$, and the left-hand side vanishes because $j - i = (u + 1)/2$. Hence

$$r'(r' - 1) + d' = s(s - 1) + 2d - 1, \tag{F}$$

$$d'(2r' + d') = d(2s - d) + 2. \tag{E}$$

Step 2. Let $\varepsilon = 2d - 1 - d'$, so that (F) reads $r'(r' - 1) = s(s - 1) + \varepsilon$.

Case $\varepsilon \geq 0$. Since $d \leq s$ and $d' \geq 1$, we have $\varepsilon \leq 2s - 2$, hence $s(s - 1) \leq r'(r' - 1) < s(s + 1)$. As $s(s - 1) \geq 2$, this forces $r' \geq 2$; the map $x \mapsto x(x - 1)$ is strictly increasing on the integers $x \geq 1$, so $r' = s$ and $\varepsilon = 0$, that is, $d' = 2d - 1$. Substituting in (E),

$$(2d - 1)(2s + 2d - 1) - d(2s - d) - 2 = (d - 1)(2s + 5d + 1) = 0,$$

so $d = 1$. Then $r = s - 1$ and $u = 2s - 1$, so $s = n + 1$ and $i = T_{n+1} - 1 - n = T_n$; finally $j = T_n + n + 1 = T_{n+1}$.

Case $\varepsilon < 0$. Then $r'(r' - 1) < s(s - 1)$, so $r' \leq s - 1$ and $r'(r' - 1) \leq (s - 1)(s - 2)$. By (F),

$$d' = s(s - 1) + 2d - 1 - r'(r' - 1) \geq 2(s - 1) + 2d - 1 \geq 2s - 1.$$

On the other hand, by (E), $d'^2 \leq d'(2r' + d') = s^2 - r^2 + 2 \leq s^2 + 2 < (s + 1)^2$, so $d' \leq s$. Hence $2s - 1 \leq s$, that is, $s \leq 1$, a contradiction. $\square$

4 The equivalence

Theorem 1. `0eisA105020.conjecture` holds if and only if every even integer greater than 2 is the sum of two primes.

Proof. Let $n \geq 1$. The indices $i = T_n$ and $j = T_{n+1}$ satisfy the hypotheses of the conjecture: $a(T_n) = 2n + 1$ and $a(T_{n+1}) = 2n + 3$ by Lemma 2, and $T_{n+1} = T_n + n + 1$. The indices strictly between them are $T_n + k$ with $1 \leq k \leq n$, and by Lemma 2

$$a(T_n + k) = (k + 1)(2n + 1 - k) = pq, \quad p = k + 1, \quad q = 2n + 1 - k.$$

Then $p + q = 2n + 2$ and $p, q \geq 2$. Since $\Omega(pq) = \Omega(p) + \Omega(q)$ and both terms are at least 1, the number pq is a semiprime if and only if p and q are both prime.

($\Rightarrow$) Let $m > 2$ be even, and write $m = 2n + 2$ with $n \geq 1$. Applying the conjecture to (n, T_n, T_{n+1}) gives an index strictly between T_n and T_{n+1} , that is, $T_n + k$ with $1 \leq k \leq n$, at which a is a semiprime; hence $m = p + q$ with p and q prime.

($\Leftarrow$) Let (n, i, j) satisfy the hypotheses. By Lemma 3, $i = T_n$ and $j = T_{n+1}$. By Goldbach, $2n + 2 = p + q$ with primes $p \leq q$, so $2 \leq p \leq n + 1$. The index $T_n + (p - 1)$ lies strictly between i and j , and $a(T_n + (p - 1)) = p(2n + 2 - p) = pq$ is a semiprime. $\square$

Without Lemma 3 one obtains only that the formal statement implies Goldbach. The formal statement quantifies over all pairs (i, j) satisfying the hypotheses, and Lemma 3 is what yields the converse.

5 Lean formalization and verification

5.1 What is formalized

The file `Solution.lean` (Appendix A, 259 lines) proves the nine statements collected in `Challenge.lean` (Appendix B), where each proof is `sorry`. Together they cover every result stated in Sections 2–4 (all names are in the namespace `A105020Goldbach`):

Result	Lean statements
Lemma 1	<code>exists_index_decomposition</code> <code>index_decomposition_unique</code>
Lemma 2	<code>antidiagonalIndex_triangularNumber_add</code> <code>a_triangularNumber_add</code> <code>a_eq_d_mul</code> <code>a_triangularNumber</code>
Remark after Lemma 2	<code>a_eq_iff_exists_factorization</code>
Lemma 3	<code>hypotheses_force_canonical_indices</code>
Theorem 1	<code>conjecture_iff_goldbach</code>

The definitions are the repository's own, and both conjectures enter only through `type_of%`, so neither statement is restated:

```
theorem conjecture_iff_goldbach :
  (type_of% @OeisA105020.conjecture)  $\leftrightarrow$  (type_of% @GoldbachConjecture.goldbach)
```

The type of `GoldbachConjecture.goldbach` is `True  $\leftrightarrow$  P`, where `P` is the Goldbach statement, so the theorem states the first line below, which is equivalent to the second:

```
OeisA105020.conjecture  $\leftrightarrow$  (True  $\leftrightarrow$  P)
OeisA105020.conjecture  $\leftrightarrow$  P
```

5.2 Verification

The formalization was checked with comparator [4], configured as in Appendix C. For each listed theorem, comparator guarantees that the solution module (i) proves the same elaborated statement as the challenge module, (ii) uses no axioms beyond `propext`, `Classical.choice` and `Quot.sound`, and (iii) is accepted by the Lean kernel.

Result. Your solution is okay! for all nine theorems (98 s).

Component	Version
formal-conjectures	commit <code>0a8b856c01b3911c52174ea4d0944097bd398bf7</code> (2026-09-10), with the <code>lakefile.toml</code> change described in Section 7
Lean	v4.33.1
Mathlib	commit <code>0df444a360eaa60ab8c11dca51a86af692955474</code> (tag v4.33.1)
comparator	commit <code>2312244ac716564a61cc0bf4e107d9abf1757a61</code> (2026-08-30)
lean4export	commit <code>411dce7db58a3afc60ecab2d211acd1042b593dc</code> (2026-08-25)
Platform	macOS; <code>landrun</code> replaced by a non-sandboxing shim (Section 7)

We ran comparator through a local wrapper that also performs a syntactic pre-check of the solution (no `sorry`, `admit`, `axiom` or `native_decide`; imports restricted to Mathlib and the two repository modules), runs the build under macOS `sandbox-exec`, and checks that the repository files are unchanged after the run. None of this is needed to reproduce the result.

5.3 Meaning of the challenge

comparator certifies that the solution proves exactly the statements of the challenge; it does not certify that those statements mean what this note says. That is checked by `MeaningChecks.lean` (Appendix D), which compiles against the same repository using only standard axioms:

- Both repository statements elaborate, by `rfl`, to the forms quoted in Section 1. In particular the Goldbach side is definitionally `True ↔ P` (and not `False ↔ P`), and `goldbach_statement_elaborates` depends only on `propext`: the elaborated statement contains no `sorryAx`.
- `Prime`, `Even` and `Nat.IsSemiprime` have their usual meaning on $\mathbb{N}$.
- `triangularNumber` agrees with T_c , and `a` agrees with the OEIS b-file for $a(0), \dots, a(11)$; the terms of Hiebl’s example sit at the indices 6, $\dots$, 10.
- The hypotheses of Lemma 3 are satisfiable ($n = 3$, $i = T_3$, $j = T_4$), so that statement is not vacuous.
- Each challenge statement is restated as a proposition, generated from the text of `Challenge.lean`, and printed with full names, which shows that `a`, `triangularNumber` and `antidiagonalIndex` resolve to the repository’s constants in the namespace `0e1sA105020`.

5.4 Remarks

- The formal proof follows Sections 2–4. In Lemma 3 it distinguishes the cases $r' < s$, $r' = s$ and $r' > s$ instead of the sign of ε ; the two case splits cover the same ground.
- Nonlinear steps are discharged by `linear_combination` and `linarith` from explicitly stated polynomial identities, each checked by `ring`. No options such as `maxHeartbeats` are set.
- As a sanity check independent of the proof, an exhaustive search found no index pair other than (T_n, T_{n+1}) satisfying the hypotheses of Lemma 3, among all $i < 10^{10}$ and among all non-canonical indices whose value is at most $4 \cdot 10^9$.

6 Prior work, and what this note adds

The mathematical fact is not new. OEIS A045917 [3], the number of decompositions of $2n$ into unordered sums of two primes, contains the formula

$$A045917(n) = \sum_{k=n(n-1)/2+2}^{n(n+1)/2} A064911(A105020(k-1))$$

(W. I. Hurt, Sep 11 2021), which in the entry, where $a(n)$ stands for A045917(n), reads

`a(n) = Sum_{k=n*(n-1)/2+2..n*(n+1)/2} A064911(A105020(k-1))`

Here A064911 [5] is the characteristic function of the semiprimes. The indices $k-1$ run over $T_{n-1}+1, \dots, T_n-1$, that is, over positions $1, \dots, n-1$ of antidiagonal $n-1$ of A105020, which are the terms strictly between the odd terms $2n-1$ and $2n+1$. The formula therefore says that the number of semiprimes strictly between these consecutive odd terms equals the number of Goldbach partitions of $2n$, and the equivalence of Hiebl's conjecture with Goldbach at the canonical indices follows at once. We checked the formula numerically for $2 \leq n \leq 1500$. Two further entries by the same author rest on the same correspondence: A228553 [6] and A350419 [7]. The underlying identity $(n+1)^2 - m^2 = (n+1-m)(n+1+m)$, which turns Goldbach partitions of $2n+2$ into semiprimes written as differences of squares, is classical.

Not ours. The correspondence between semiprimes in A105020 and Goldbach partitions, and with it the equivalence at the canonical indices.

What this note contributes.

1. The observation that the repository's formal statement `0eisA105020.conjecture`, which quantifies over arbitrary indices, is equivalent to `GoldbachConjecture.goldbach`. This requires Lemma 3. We did not find Lemma 3 written elsewhere; it is a routine computation, and we make no claim that it is new.
2. A Lean 4 formalization of every result stated in Sections 2–4, checked with comparator, together with the meaning checks of Section 5.3.

Provenance. The observation that `0eisA105020.conjecture` implies Goldbach was first produced by an AI language model (Claude Opus 5, Anthropic) during an exploratory run on open problems from formal-conjectures. The converse direction (Lemma 3), the proofs in this note and the Lean formalization were produced in working sessions with the same model under the author's direction, and were checked as described in Section 5. This note was drafted with AI assistance.

7 Reproducing the verification

The four files reproduced in Appendices A–D are attached to this PDF as embedded files. Most PDF viewers list them in an attachments panel, and `pdfdetach -saveall` (from Poppler) extracts them. Use the attached files rather than text copied from the appendices: copying from a PDF loses indentation, which is significant in Lean.

7.1 Repository and build configuration

```
git clone https://github.com/google-deepmind/formal-conjectures.git
cd formal-conjectures
git checkout 0a8b856c01b3911c52174ea4d0944097bd398bf7
```

Build-configuration caveat. At this commit `lakefile.toml` declares, besides the library `FormalConjectures`, a second library `FormalConjecturesAnswerPostpone` with the same glob and the option `google.answer = "postpone"`. The two libraries write to the same build directory, so whether `answer(sorry)` elaborates to `True` or to a `sorry` depends on which build command ran last. If `GoldbachConjecture.goldbach` is built in `postpone` mode, its elaborated type contains `sorryAx`, and the checks of Section 5 fail at the axiom stage. For our verification we disabled the second library by commenting out its `[[lean_lib]]` block; this is the only change to tracked files. Then:

```
lake exe cache get
lake build FormalConjectures.OEIS.«105020» \
  FormalConjectures.Wikipedia.GoldbachConjecture
```

7.2 Quick check and meaning checks, without comparator

Save `Solution.lean` as `FormalConjectures/A105020Goldbach/Solution.lean`, append lines such as

```
#print axioms A105020Goldbach.conjecture_iff_goldbach
```

(one for each theorem), and run `lake env lean` on the file. Every theorem should depend only on standard axioms, among `propext`, `Classical.choice` and `Quot.sound`. The repository's style linters may print warnings, for instance about importing `FormalConjecturesUtil`; they do not affect the result. This shows that the proofs contain no `sorry`, but not that the statements are the intended ones.

For the latter, save `MeaningChecks.lean` as `FormalConjectures/A105020Goldbach/MeaningChecks.lean` and run `lake env lean` on it. All declarations should compile, the printed statements should mention the constants of `OeisA105020`, and the final lines should report standard axioms only, with `goldbach_statement_elaborates` depending on `propext` alone.

7.3 Full check with comparator

1. Build `lean4export` at the commit above with Lean v4.33.1, and comparator at the commit above, following their READMEs. On Linux, install `landrun` as described in the comparator README.
2. Save `Challenge.lean` and `Solution.lean` in `FormalConjectures/A105020Goldbach/`, and `config.json` in the repository root.
3. Build the challenge with `lake build FormalConjectures.A105020Goldbach.Challenge`. Per the comparator README, the challenge is trusted, and the solution should not be built beforehand outside comparator.
4. Run comparator as in its README, with `lean4export` and `landrun` in `PATH` (or with `COMPARATOR_LEAN4EXPORT` and `COMPARATOR_LANDRUN` set):

```
systemd-run --property=RestrictAddressFamilies=~AF_UNIX --user --pty -E PATH="$PATH" \
--working-directory "$(pwd)" -- bash -c 'lake env /path/to/comparator config.json'
```

The expected final line is `Your solution is okay!`.

On macOS `landrun` is not available. `comparator` provides `scripts/fake-landrun.sh`, which runs the build without a process sandbox; we used an equivalent shim. The logical guarantees of Section 5 are unaffected. What is lost is protection against a solution file that executes arbitrary code during compilation.

References

- [1] Google DeepMind, *formal-conjectures*,
<https://github.com/google-deepmind/formal-conjectures>, commit `0a8b856c`.
- [2] OEIS Foundation Inc., *The On-Line Encyclopedia of Integer Sequences*, sequence A105020,
<https://oeis.org/A105020>, accessed 12 September 2026.
- [3] OEIS, sequence A045917, <https://oeis.org/A045917>.
- [4] *comparator*, <https://github.com/leanprover/comparator>, commit `2312244a`.
- [5] OEIS, sequence A064911, <https://oeis.org/A064911>.
- [6] OEIS, sequence A228553, <https://oeis.org/A228553>.
- [7] OEIS, sequence A350419, <https://oeis.org/A350419>.
- [8] The mathlib Community, *The Lean mathematical library*,
<https://github.com/leanprover-community/mathlib4>, commit `0df444a3`.

A Solution.lean

```
import FormalConjectures.OEIS.«105020»
import FormalConjectures.Wikipedia.GoldbachConjecture

/-!
# OEIS A105020 and the binary Goldbach conjecture

Proof that `OeisA105020.conjecture` is equivalent to `GoldbachConjecture.goldbach`,
both taken verbatim from the formal-conjectures repository via `type_of%`.
Definitions used: `OeisA105020.a`, `triangularNumber`, `antidiagonalIndex`.
-/

namespace A105020Goldbach

open OeisA105020

/--  $2 \cdot T\ c = c(c + 1)$ . -/
lemma two_mul_triangularNumber (c : ℕ) : 2 * triangularNumber c = c * (c + 1) := by
  have h2 : 2 ∣ (c + 1) * c := by
    have := Nat.even_mul_succ_self c
    rw [Nat.mul_comm] at this
    exact even_iff_two_dvd.mp this
  unfold triangularNumber
  rw [Nat.choose_two_right, Nat.add_sub_cancel, Nat.mul_div_cancel' h2, Nat.mul_comm]
```

```

lemma triangularNumber_succ (n : N) : triangularNumber (n + 1) = triangularNumber n + (n + 1)
  ↪ := by
  have h1 := two_mul_triangularNumber n
  have h2 := two_mul_triangularNumber (n + 1)
  have h3 : (n + 1) * (n + 1 + 1) = n * (n + 1) + 2 * (n + 1) := by ring
  omega

/-- Every index lies between `T c` and `T c + c`, where `c` is its antidiagonal. -/
lemma triangularNumber_le_and_le (N : N) :
  triangularNumber (antidiagonalIndex N) ≤ N ∧
  N ≤ triangularNumber (antidiagonalIndex N) + antidiagonalIndex N := by
  have h1 := Nat.sqrt_le (8 * N + 1)
  have h2 := Nat.lt_succ_sqrt (8 * N + 1)
  have ht : 1 ≤ Nat.sqrt (8 * N + 1) := Nat.le_sqrt.mpr (by omega)
  have hc : antidiagonalIndex N = (Nat.sqrt (8 * N + 1) - 1) / 2 := rfl
  set t := Nat.sqrt (8 * N + 1) with ht_def
  set c := antidiagonalIndex N with hc_def
  have hT := two_mul_triangularNumber c
  have hlo : 2 * c + 1 ≤ t := by omega
  have hhi : t ≤ 2 * c + 2 := by omega
  simp only [Nat.succ_eq_add_one] at h2
  constructor
  · nlinarith
  · nlinarith

/-- The indices on antidiagonal `c` are `T c + k` with `k ≤ c`. -/
lemma antidiagonalIndex_triangularNumber_add {c k : N} (hk : k ≤ c) :
  antidiagonalIndex (triangularNumber c + k) = c := by
  have hT := two_mul_triangularNumber c
  have hlo : 2 * c + 1 ≤ Nat.sqrt (8 * (triangularNumber c + k) + 1) := by
    rw [Nat.le_sqrt]; nlinarith
  have hhi : Nat.sqrt (8 * (triangularNumber c + k) + 1) < 2 * c + 3 := by
    rw [Nat.sqrt_lt]; nlinarith
  show (Nat.sqrt (8 * (triangularNumber c + k) + 1) - 1) / 2 = c
  omega

/-- Closed form: `a (T c + k) = (k + 1)(2c + 1 - k)` for `k ≤ c`. -/
theorem a_triangularNumber_add {c k : N} (hk : k ≤ c) :
  a (triangularNumber c + k) = (k + 1) * (2 * c + 1 - k) := by
  simp only [a]
  rw [antidiagonalIndex_triangularNumber_add hk, Nat.add_sub_cancel_left]
  obtain ⟨m, rfl⟩ := Nat.exists_eq_add_of_le hk
  have e1 : k + m - k = m := by omega
  have e2 : 2 * (k + m) + 1 - k = k + 2 * m + 1 := by omega
  rw [e1, e2]
  have e3 : (k + m + 1) ^ 2 = m ^ 2 + (k + 1) * (k + 2 * m + 1) := by ring
  rw [e3, Nat.add_sub_cancel_left]

/-- **Index decomposition**, existence: every `N` is `T c + k` with `k ≤ c`. -/
theorem exists_index_decomposition (N : N) : ∃ c k, k ≤ c ∧ N = triangularNumber c + k := by
  obtain ⟨h1, h2⟩ := triangularNumber_le_and_le N
  exact ⟨antidiagonalIndex N, N - triangularNumber (antidiagonalIndex N), by omega, by omega⟩

/-- **Index decomposition**, uniqueness. -/
theorem index_decomposition_unique {c k c' k' : N} (hk : k ≤ c) (hk' : k' ≤ c')
  (h : triangularNumber c + k = triangularNumber c' + k') : c = c' ∧ k = k' := by
  have e : c = c' := by rw [← antidiagonalIndex_triangularNumber_add hk, h,
    ↪ antidiagonalIndex_triangularNumber_add hk']
  subst e
  exact ⟨rfl, by omega⟩

/-- **The formula `a(N) = d (2s - d)`**, with `s = c + 1` and `d = k + 1`. -/
theorem a_eq_d_mul {c k : N} (hk : k ≤ c) :

```

```

    a (triangularNumber c + k) = (k + 1) * (2 * (c + 1) - (k + 1)) := by
  rw [a_triangularNumber_add hk]
  congr 1
  omega

lemma a_triangularNumber (n : ℕ) : a (triangularNumber n) = 2 * n + 1 := by
  simp using a_triangularNumber_add (Nat.zero_le n)

/-- **Values and factorizations**: `a N = v` exactly when `v = d * e` with `1 ≤ d ≤ e`,
`d ≡ e (mod 2)`, and `N = T ((d + e) / 2 - 1) + (d - 1)`. -/
theorem a_eq_iff_exists_factorization (N v : ℕ) :
  a N = v ↔ ∃ d e, 1 ≤ d ∧ d ≤ e ∧ d % 2 = e % 2 ∧ v = d * e ∧
  N = triangularNumber ((d + e) / 2 - 1) + (d - 1) := by
  constructor
  · intro h
    obtain ⟨c, k, hk, rfl⟩ := exists_index_decomposition N
    rw [a_triangularNumber_add hk] at h
    refine ⟨k + 1, 2 * c + 1 - k, by omega, by omega, by omega, h.symm, ?_⟩
    have e1 : (k + 1 + (2 * c + 1 - k)) / 2 - 1 = c := by omega
    rw [e1]
    omega
  · rintro ⟨d, e, hd, hde, hpar, rfl, rfl⟩
    have hk : d - 1 ≤ (d + e) / 2 - 1 := by omega
    rw [a_triangularNumber_add hk]
    have e1 : d - 1 + 1 = d := by omega
    have e2 : 2 * ((d + e) / 2 - 1) + 1 - (d - 1) = e := by omega
    rw [e1, e2]

/-- **Index lemma**: the hypotheses of the conjecture force the canonical indices. -/
theorem hypotheses_force_canonical_indices {n i j : ℕ} (hn : 1 ≤ n) (hi : a i = 2 * n + 1)
(hj : a j = 2 * n + 3) (hij : j = i + n + 1) :
  i = triangularNumber n ∧ j = triangularNumber (n + 1) := by
  obtain ⟨c, k, hk, rfl⟩ := exists_index_decomposition i
  obtain ⟨c', k', hk', rfl⟩ := exists_index_decomposition j
  obtain ⟨m, rfl⟩ := Nat.exists_eq_add_of_le hk
  obtain ⟨m', rfl⟩ := Nat.exists_eq_add_of_le hk'
  rw [a_triangularNumber_add hk] at hi
  rw [a_triangularNumber_add hk'] at hj
  have e1 : 2 * (k + m) + 1 - k = k + 2 * m + 1 := by omega
  have e2 : 2 * (k' + m') + 1 - k' = k' + 2 * m' + 1 := by omega
  rw [e1] at hi
  rw [e2] at hj
  have hkm : 1 ≤ k + m := by
    rcases Nat.eq_zero_or_pos (k + m) with h | h
    · have hk0 : k = 0 := by omega
      have hm0 : m = 0 := by omega
      rw [hk0, hm0] at hi
      norm_num at hi
      omega
    · exact h
  have t1 := two_mul_triangularNumber (k + m)
  have t2 := two_mul_triangularNumber (k' + m')
  have Hi : ((k : ℤ) + 1) * (k + 2 * m + 1) = 2 * n + 1 := by exact_mod_cast hi
  have Hj : ((k' : ℤ) + 1) * (k' + 2 * m' + 1) = 2 * n + 3 := by exact_mod_cast hj
  have Hij : (triangularNumber (k' + m') : ℤ) + k' =
    (triangularNumber (k + m) : ℤ) + k + n + 1 := by exact_mod_cast hij
  have T1 : 2 * (triangularNumber (k + m) : ℤ) = ((k : ℤ) + m) * (k + m + 1) := by
    exact_mod_cast t1
  have T2 : 2 * (triangularNumber (k' + m') : ℤ) = ((k' : ℤ) + m') * (k' + m' + 1) := by
    exact_mod_cast t2
  -- (F) and (E) of the written proof, with r = m, r' = m', d = k + 1, d' = k' + 1
  have F : (m' : ℤ) * m' + k' = ((k : ℤ) + m) * (k + m + 1) + 2 * k + m' := by
    linear_combination 2 * Hij - T2 + T1 - Hj

```

```

have E : ((k' : Z) + 1) * (k' + 2 * m' + 1) = ((k : Z) + 1) * (k + 2 * m + 1) + 2 := by
  linear_combination Hj - Hi
have hk0' : (0 : Z) ≤ k := by positivity
have hm0' : (0 : Z) ≤ m := by positivity
have hk'0 : (0 : Z) ≤ k' := by positivity
have hm'0 : (0 : Z) ≤ m' := by positivity
have hkmZ : (1 : Z) ≤ k + m := by exact_mod_cast hkm
have hk_zero : k = 0 := by
  rcases lt_trichotomy m' (k + m + 1) with h | h | h
  · -- case r' ≤ s - 1: impossible
    exfalse
    have hle : (m' : Z) ≤ k + m := by
      have : m' ≤ k + m := by omega
      exact_mod_cast this
    have hprod : (0 : Z) ≤ ((k : Z) + m - m') * (k + m + m' - 1) :=
      mul_nonneg (by linarith) (by linarith)
    have hexp : ((k : Z) + m - m') * (k + m + m' - 1)
      = ((k : Z) + m) * (k + m + 1) - m' * m' - 2 * (k + m) + m' := by ring
    have hk'ge : (4 * k + 2 * m : Z) ≤ k' := by linarith
    have h1 : (4 * k + 2 * m + 1 : Z) ≤ k' + 1 := by linarith
    have h2 : (k' + 1 : Z) ≤ k' + 2 * m' + 1 := by linarith
    have hsq : (4 * k + 2 * m + 1 : Z) * (4 * k + 2 * m + 1)
      ≤ ((k' : Z) + 1) * (k' + 2 * m' + 1) :=
      mul_le_mul h1 (h1.trans h2) (by linarith) (by linarith)
    have hsq2 : ((k : Z) + m) * 1 ≤ (k + m) * (k + m) :=
      mul_le_mul_of_nonneg_left hkmZ (by linarith)
    have hkk : (0 : Z) ≤ k * k := mul_nonneg hk0' hk0'
    have hkm' : (0 : Z) ≤ k * m := mul_nonneg hk0' hm0'
    linarith
  · -- case r' = s: forces d = 1
    have hZ : (m' : Z) = k + m + 1 := by exact_mod_cast h
    rw [hZ] at F E
    have hk'2 : (k' : Z) = 2 * k := by linear_combination F
    rw [hk'2] at E
    have hprod : (k : Z) * (7 * k + 2 * m + 8) = 0 := by linear_combination E
    have hpos : (0 : Z) ≤ k * (7 * k + 2 * m) := mul_nonneg hk0' (by linarith)
    have hsplit : (k : Z) * (7 * k + 2 * m + 8) = k * (7 * k + 2 * m) + 8 * k := by ring
    have hk0 : (k : Z) = 0 := by linarith
    exact_mod_cast hk0
  · -- case r' ≥ s + 1: impossible
    exfalse
    have hge : (k : Z) + m + 2 ≤ m' := by
      have : k + m + 2 ≤ m' := by omega
      exact_mod_cast this
    have hprod : (0 : Z) ≤ ((m' : Z) - (k + m + 2)) * (m' + k + m + 1) :=
      mul_nonneg (by linarith) (by linarith)
    have hexp : ((m' : Z) - (k + m + 2)) * (m' + k + m + 1)
      = m' * m' - m' - ((k : Z) + m) * (k + m + 1) - 2 * (k + m + 1) := by ring
    linarith
subst hk_zero
have hmn : m = n := by
  norm_num at hi
  omega
subst hmn
constructor
· simp
· rw [triangularNumber_succ]
  simp only [Nat.zero_add, Nat.add_zero] at hij
  omega

/-- If `p * q` is a semiprime and `p, q ≥ 2`, then both factors are prime. -/
lemma primes_of_semiprime_mul {p q : ℕ} (hp : 2 ≤ p) (hq : 2 ≤ q)
(h : (p * q).IsSemiprime) : p.Prime ∧ q.Prime := by

```

```

obtain ⟨_, h2⟩ := h
rw [ArithmeticFunction.cardFactors_mul (by omega) (by omega)] at h2
have hp1 : ArithmeticFunction.cardFactors p ≠ 0 := by
  rw [Ne, ArithmeticFunction.cardFactors_eq_zero_iff_eq_zero_or_one]; omega
have hq1 : ArithmeticFunction.cardFactors q ≠ 0 := by
  rw [Ne, ArithmeticFunction.cardFactors_eq_zero_iff_eq_zero_or_one]; omega
exact ⟨ArithmeticFunction.cardFactors_eq_one_iff_prime.mp (by omega),
  ArithmeticFunction.cardFactors_eq_one_iff_prime.mp (by omega)⟩

/-- **The equivalence**, between the two repository statements taken verbatim. -/
theorem conjecture_iff_goldbach :
  (type_of% @OeisA105020.conjecture) ↔ (type_of% @GoldbachConjecture.goldbach) := by
show (∀ (n i j : ℕ), 1 ≤ n → a i = 2 * n + 1 → a j = 2 * n + 3 → j = i + n + 1 →
  ∃ k, i < k ∧ k < j ∧ (a k).IsSemiprime) ↔
  (True ↔ ∀ n : ℕ, 2 < n → Even n → ∃ p q, Prime p ∧ Prime q ∧ n = p + q)
rw [true_iff]
constructor
· intro H m hm hev
  obtain ⟨r, hr⟩ := hev
  obtain ⟨n, rfl⟩ : ∃ n, m = 2 * n + 2 := ⟨r - 1, by omega⟩
  have hn : 1 ≤ n := by omega
  obtain ⟨k, hk1, hk2, hsp⟩ := H n (triangularNumber n) (triangularNumber (n + 1)) hn
    (a_triangularNumber n) (a_triangularNumber (n + 1)) (triangularNumber_succ n)
  rw [triangularNumber_succ] at hk2
  obtain ⟨κ, rfl⟩ := Nat.exists_eq_add_of_lt hk1
  have hk : κ + 1 ≤ n := by omega
  rw [show triangularNumber n + κ + 1 = triangularNumber n + (κ + 1) by omega,
    a_triangularNumber_add hk] at hsp
  have hq : 2 * n + 1 - (κ + 1) = 2 * n - κ := by omega
  rw [hq] at hsp
  obtain ⟨hp', hq'⟩ := primes_of_semiprime_mul (by omega) (by omega) hsp
  exact ⟨κ + 1 + 1, 2 * n - κ, Nat.prime_iff.mp hp', Nat.prime_iff.mp hq', by omega⟩
· intro G n i j hn hi hj hij
  obtain ⟨rfl, rfl⟩ := hypotheses_force_canonical_indices hn hi hj hij
  obtain ⟨p, q, hp, hq, hpq⟩ := G (2 * n + 2) (by omega) ⟨n + 1, by ring⟩
  rw [← Nat.prime_iff] at hp hq
  wlog hle : p ≤ q generalizing p q
  · exact this q p hq hp (by omega) (by omega)
  have hp2 := hp.two_le
  refine ⟨triangularNumber n + (p - 1), by omega, ?_, ?_⟩
  · rw [triangularNumber_succ]; omega
  · rw [a_triangularNumber_add (by omega)]
    have e : 2 * n + 1 - (p - 1) = q := by omega
    rw [e, show p - 1 + 1 = p by omega]
    exact Nat.Prime.mul_isAlmostPrime_two hp hq
end A105020Goldbach

```

B Challenge.lean

```

import FormalConjectures.OEIS.«105020»
import FormalConjectures.Wikipedia.GoldbachConjecture

/-!
# Challenge: OEIS A105020 and the binary Goldbach conjecture

Statements only; every proof is `sorry`. comparator checks that the solution proves
exactly these statements. `triangularNumber`, `antidiagonalIndex`, `a`,
`OeisA105020.conjecture` and `GoldbachConjecture.goldbach` are the repository's own.
-!

```

```

namespace A105020Goldbach

open OeisA105020

/-- Lemma 1: every index is `T c + k` with `k ≤ c`, ... -/
theorem exists_index_decomposition (N : ℕ) :
  ∃ c k, k ≤ c ∧ N = triangularNumber c + k := by
  sorry

/-- Lemma 1: ... in exactly one way, ... -/
theorem index_decomposition_unique {c k c' k' : ℕ} (hk : k ≤ c) (hk' : k' ≤ c')
  (h : triangularNumber c + k = triangularNumber c' + k') : c = c' ∧ k = k' := by
  sorry

/-- Lemma 1: ... and then `c` is the antidiagonal index. -/
theorem antidiagonalIndex_triangularNumber_add {c k : ℕ} (hk : k ≤ c) :
  antidiagonalIndex (triangularNumber c + k) = c := by
  sorry

/-- Lemma 2: `a (T c + k) = (k + 1)(2c + 1 - k)` for `k ≤ c`. -/
theorem a_triangularNumber_add {c k : ℕ} (hk : k ≤ c) :
  a (triangularNumber c + k) = (k + 1) * (2 * c + 1 - k) := by
  sorry

/-- Lemma 2: `a(N) = d (2s - d)` with `s = c + 1` and `d = k + 1`. -/
theorem a_eq_d_mul {c k : ℕ} (hk : k ≤ c) :
  a (triangularNumber c + k) = (k + 1) * (2 * (c + 1) - (k + 1)) := by
  sorry

/-- Lemma 2: `a (T n) = 2n + 1`. -/
theorem a_triangularNumber (n : ℕ) : a (triangularNumber n) = 2 * n + 1 := by
  sorry

/-- Remark after Lemma 2: the indices where `a` takes a given value. -/
theorem a_eq_iff_exists_factorization (N v : ℕ) :
  a N = v ↔ ∃ d e, 1 ≤ d ∧ d ≤ e ∧ d % 2 = e % 2 ∧ v = d * e ∧
  N = triangularNumber ((d + e) / 2 - 1) + (d - 1) := by
  sorry

/-- Lemma 3: the hypotheses of the conjecture force the canonical indices. -/
theorem hypotheses_force_canonical_indices {n i j : ℕ} (hn : 1 ≤ n)
  (hi : a i = 2 * n + 1) (hj : a j = 2 * n + 3) (hij : j = i + n + 1) :
  i = triangularNumber n ∧ j = triangularNumber (n + 1) := by
  sorry

/-- Theorem: the equivalence between the two repository statements. -/
theorem conjecture_iff_goldbach :
  (type_of% @OeisA105020.conjecture) ↔ (type_of% @GoldbachConjecture.goldbach) := by
  sorry

end A105020Goldbach

```

C config.json

```

{
  "challenge_module": "FormalConjectures.A105020Goldbach.Challenge",
  "solution_module": "FormalConjectures.A105020Goldbach.Solution",
  "theorem_names": [
    "A105020Goldbach.exists_index_decomposition",
  ]
}

```

```

    "A105020Goldbach.index_decomposition_unique",
    "A105020Goldbach.antidiagonalIndex_triangularNumber_add",
    "A105020Goldbach.a_triangularNumber_add",
    "A105020Goldbach.a_eq_d_mul",
    "A105020Goldbach.a_triangularNumber",
    "A105020Goldbach.a_eq_iff_exists_factorization",
    "A105020Goldbach.hypotheses_force_canonical_indices",
    "A105020Goldbach.conjecture_iff_goldbach"
  ],
  "permitted_axioms": [
    "propext",
    "Quot.sound",
    "Classical.choice"
  ]
}

```

D MeaningChecks.lean

```

import FormalConjectures.OEIS.«105020»
import FormalConjectures.Wikipedia.GoldbachConjecture

/-!
# Meaning checks for the A105020 / Goldbach challenge

comparator certifies that `Solution.lean` proves exactly the statements of
`Challenge.lean`. This file checks that those statements mean what the note says.
Every declaration must compile, and the `#print axioms` lines at the end must report
only standard axioms.

Section 1 is generated from `Challenge.lean` by `make_meaning_checks.py`.
-/

namespace A105020Goldbach.MeaningChecks

open OeisA105020

/-! ## 1. The challenge statements, printed with full names -/

def P_exists_index_decomposition : Prop := ∀ (N : ℕ), ∃ c k, k ≤ c ∧ N = triangularNumber c + k
def P_index_decomposition_unique : Prop := ∀ {c k c' k' : ℕ} (hk : k ≤ c) (hk' : k' ≤ c')
  (h : triangularNumber c + k = triangularNumber c' + k'), c = c' ∧ k = k'
def P_antidiagonalIndex_triangularNumber_add : Prop := ∀ {c k : ℕ} (hk : k ≤ c),
  ↔ antidiagonalIndex (triangularNumber c + k) = c
def P_a_triangularNumber_add : Prop := ∀ {c k : ℕ} (hk : k ≤ c), a (triangularNumber c + k) = (
  ↔ k + 1) * (2 * c + 1 - k)
def P_a_eq_d_mul : Prop := ∀ {c k : ℕ} (hk : k ≤ c), a (triangularNumber c + k) = (k + 1) * (2 *
  ↔ (c + 1) - (k + 1))
def P_a_triangularNumber : Prop := ∀ (n : ℕ), a (triangularNumber n) = 2 * n + 1
def P_a_eq_iff_exists_factorization : Prop := ∀ (N v : ℕ), a N = v ↔ ∃ d e, 1 ≤ d ∧ d ≤ e ∧ d %
  ↔ 2 = e % 2 ∧ v = d * e ∧ N = triangularNumber ((d + e) / 2 - 1) + (d - 1)
def P_hypotheses_force_canonical_indices : Prop := ∀ {n i j : ℕ} (hn : 1 ≤ n)
  (hi : a i = 2 * n + 1) (hj : a j = 2 * n + 3) (hij : j = i + n + 1), i = triangularNumber n
  ↔ ∧ j = triangularNumber (n + 1)
def P_conjecture_iff_goldbach : Prop := (type_of% @OeisA105020.conjecture) ↔ (type_of%
  ↔ @GoldbachConjecture.goldbach)

end A105020Goldbach.MeaningChecks

set_option pp.fullNames true in
set_option pp.numericTypes true in

```

```

#print A105020Goldbach.MeaningChecks.P_exists_index_decomposition
set_option pp.fullNames true in
set_option pp.numericTypes true in
#print A105020Goldbach.MeaningChecks.P_index_decomposition_unique
set_option pp.fullNames true in
set_option pp.numericTypes true in
#print A105020Goldbach.MeaningChecks.P_antidiagonalIndex_triangularNumber_add
set_option pp.fullNames true in
set_option pp.numericTypes true in
#print A105020Goldbach.MeaningChecks.P_a_triangularNumber_add
set_option pp.fullNames true in
set_option pp.numericTypes true in
#print A105020Goldbach.MeaningChecks.P_a_eq_d_mul
set_option pp.fullNames true in
set_option pp.numericTypes true in
#print A105020Goldbach.MeaningChecks.P_a_triangularNumber
set_option pp.fullNames true in
set_option pp.numericTypes true in
#print A105020Goldbach.MeaningChecks.P_a_eq_iff_exists_factorization
set_option pp.fullNames true in
set_option pp.numericTypes true in
#print A105020Goldbach.MeaningChecks.P_hypotheses_force_canonical_indices
set_option pp.fullNames true in
set_option pp.numericTypes true in
#print A105020Goldbach.MeaningChecks.P_conjecture_iff_goldbach

namespace A105020Goldbach.MeaningChecks

open OeisA105020

/-! ## 2. The two repository statements, as elaborated -/

/-- The Goldbach side is definitionally `True ↔ P`: `answer(sorry)` has become `True`,
not `False`, and not a `sorry` (see the axioms printed at the end). -/
theorem goldbach_statement_elaborates :
  (type_of% @GoldbachConjecture.goldbach) =
    (True ↔ ∀ n : ℕ, 2 < n → Even n → ∃ p q, Prime p ∧ Prime q ∧ n = p + q) := rfl

/-- The conjecture side is definitionally the statement in
`FormalConjectures/OEIS/105020.lean`. -/
theorem conjecture_statement_elaborates :
  (type_of% @OeisA105020.conjecture) =
    (∀ (n i j : ℕ), 1 ≤ n → a i = 2 * n + 1 → a j = 2 * n + 3 → j = i + n + 1 →
      ∃ k, i < k ∧ k < j ∧ (a k).IsSemiprime) := rfl

/-! ## 3. Primes, evenness and semiprimes have their usual meaning -/

theorem prime_iff_nat_prime (p : ℕ) : Prime p ↔ Nat.Prime p := Nat.prime_iff.symm

theorem even_iff_exists_add (m : ℕ) : Even m ↔ ∃ r, m = r + r := Iff.rfl

theorem isSemiprime_iff (m : ℕ) :
  m.IsSemiprime ↔ m ≠ 0 ∧ ArithmeticFunction.cardFactors m = 2 := Iff.rfl

/-! ## 4. The definitions agree with `T c` and with the OEIS b-file (offset 0) -/

theorem triangularNumber_values :
  triangularNumber 0 = 0 ∧ triangularNumber 1 = 1 ∧ triangularNumber 2 = 3 ∧
  triangularNumber 3 = 6 ∧ triangularNumber 4 = 10 := by
  decide

theorem a_values_match_b_file :
  a 0 = 1 ∧ a 1 = 3 ∧ a 2 = 4 ∧ a 3 = 5 ∧ a 4 = 8 ∧ a 5 = 9 ∧

```

```

    a 6 = 7 ^ a 7 = 12 ^ a 8 = 15 ^ a 9 = 16 ^ a 10 = 9 ^ a 11 = 16 := by
    unfold a antidiagonalIndex triangularNumber
    simp only [Nat.choose_two_right]
    norm_num

/-! ## 5. Hiebl's example under the current offset

The example in the OEIS comment (n = 3) names a(7) = 7, a(8) = 12, a(9) = 15,
a(10) = 16 and a(11) = 9, that is, offset 1. With the entry's current offset 0, which
the repository also uses, the same terms are a(6), ..., a(10): the indices are
i = 6 = T 3 and j = 10 = T 4, and j = i + 3 + 1. -/
theorem hiebl_example_at_offset_zero :
    a 6 = 2 * 3 + 1 ^ a 7 = 12 ^ a 8 = 15 ^ a 9 = 16 ^ a 10 = 2 * 3 + 3 ^
    triangularNumber 3 = 6 ^ triangularNumber 4 = 10 := by
    unfold a antidiagonalIndex triangularNumber
    simp only [Nat.choose_two_right]
    norm_num

/-! ## 6. The hypotheses of Lemma 3 are satisfiable, so the lemma is not vacuous -/

theorem lemma3_hypotheses_satisfiable :
    1 ≤ 3 ^ a (triangularNumber 3) = 2 * 3 + 1 ^ a (triangularNumber 4) = 2 * 3 + 3 ^
    triangularNumber 4 = triangularNumber 3 + 3 + 1 := by
    have h3 : triangularNumber 3 = 6 := by decide
    have h4 : triangularNumber 4 = 10 := by decide
    rw [h3, h4]
    unfold a antidiagonalIndex triangularNumber
    simp only [Nat.choose_two_right]
    norm_num

end A105020Goldbach.MeaningChecks

#print axioms A105020Goldbach.MeaningChecks.goldbach_statement_elaborates
#print axioms A105020Goldbach.MeaningChecks.conjecture_statement_elaborates
#print axioms A105020Goldbach.MeaningChecks.prime_iff_nat_prime
#print axioms A105020Goldbach.MeaningChecks.even_iff_exists_add
#print axioms A105020Goldbach.MeaningChecks.isSemiprime_iff
#print axioms A105020Goldbach.MeaningChecks.triangularNumber_values
#print axioms A105020Goldbach.MeaningChecks.a_values_match_b_file
#print axioms A105020Goldbach.MeaningChecks.hiebl_example_at_offset_zero
#print axioms A105020Goldbach.MeaningChecks.lemma3_hypotheses_satisfiable

```